

🔗 О чём вообще вся курсовая — за 30 секунд

Представь: ты работаешь в интернет-магазине. Каждый день нужно обработать тысячи фото товаров, конвертировать видеообзоры, наложить AI-ретушь на фотографии.

Это делают **серверы** — мощные компьютеры. Но у каждого сервера **ограниченная память**. Задач много, серверов мало, память не резиновая.

Твоя курсовая отвечает на один вопрос:

«Какую задачу на какой сервер назначить, чтобы сделать максимум важного и при этом не сломать серверы?»

Вот и всё. Это вся суть.

🔗 Часть 1. Базовые понятия

Что такое RAM и VRAM?

Представь сервер как повара на кухне.

- **RAM** — это его рабочий стол. Чем больше стол, тем больше задач одновременно. Если места нет — сервер падает с ошибкой.
- **VRAM** — это специальная разделочная доска для сложных блюд. Нужна для видео, нейросетей, графики. Её нельзя расширить. Закончилась — задача падает, всё.

Что такое «задачи»?

В твоей работе **8 задач**. У каждой есть **приоритет** — насколько она важна для бизнеса:

1. 🔗 Задача 7 — AI-ретушь фотографий, приоритет **10**
2. Задача 3 — конвертация видео 1080p→720p, приоритет **9**
3. Задача 1 — рекламный баннер 4K, приоритет **8**
4. Задача 6 — картинки для соцсетей, приоритет **7**
5. Задача 2 — превью для 50 товаров, приоритет **6**
6. Задача 5 — ресайз фото для мобилок, приоритет **5**
7. Задача 4 — водяные знаки на 200 фото, приоритет **4**
8. 🔗 Задача 8 — GIF из видео, приоритет **3** (наименее важная)

Что такое «серверы»?

В твоей работе **3 сервера**:

- **S1** — мощный GPU-сервер (NVIDIA A100). RAM: 2048 МБ, VRAM: 1024 МБ.
- **S2** — CPU-сервер с большой памятью. RAM: 1536 МБ, VRAM: 512 МБ.
- **S3** — гибридный сервер. RAM: 1024 МБ, VRAM: 384 МБ.

❓ Часть 2. Математическая модель

Почему задача называется GMMKP?

Это страшное слово — просто **задача о рюкзаке** в усложнённом виде.

Классический рюкзак: ты идёшь в поход, рюкзак выдержит 10 кг, есть 10 вещей. Какие взять, чтобы взять самое нужное?

Твоя задача сложнее сразу в три раза:

- **Усложнение 1 (многомерность):** у тебя два ограничения вместо одного — и RAM, и VRAM. Как рюкзак с ограничением и по весу, и по объёму одновременно.
- **Усложнение 2 (множественный выбор):** у тебя три рюкзака, а не один. Нужно выбрать, в какой именно сервер класть каждую задачу.
- **Усложнение 3 (обобщённость):** оптимизируем сразу весь кластер серверов, а не один.

Из-за этого задача стала **NP-трудной** — слишком много комбинаций, чтобы просто перебрать все. Нужен умный алгоритм.

Переменные решения — это просто выключатели

Компьютер работает с переменными x_{ij} . Не пугайся — это просто выключатели лампочек:

- $x_{31} = 1 \rightarrow$? лампочка горит: задача 3 назначена на сервер 1
- $x_{31} = 0 \rightarrow$? лампочка погашена: задача 3 не идёт на сервер 1

Всего таких выключателей: 8 задач $\times$ 3 сервера = **24 штуки**. Программа решает, какие из 24 лампочек зажечь, а какие погасить.

Целевая функция — это «набрать очки»

Смысл простой: мы хотим набрать как можно больше очков-приоритетов.

- Если задача выполняется $\rightarrow$ её приоритет идёт в копилку.
- Если нет $\rightarrow$ ноль.

Теоретический максимум: $8+6+9+4+5+7+10+3 =$ **52 очка**, если все задачи влезут. Программа нашла именно 52 — все задачи поместились. Это идеальный результат! ?

Ограничения — это правила игры

Их ровно **14 штук**:

Правило 1 — уникальность (8 штук): Каждая задача идёт только на один сервер. Задача 3 не может быть одновременно на S1 и S2 — она или там, или тут, или нигде.

Правило 2 — RAM (3 штуки): Сумма памяти всех задач на сервере не должна превышать его

ёмкость. Не влезают — не назначаем.

Правило 3 — VRAM (3 штуки): То же самое для видеопамяти. Это самое жёсткое ограничение в задаче — главный «барьер».

Итого: $8 + 3 + 3 = 14$ ограничений.

🔍 Секрет «узкого горлышка»

- Суммарная потребность всех задач в VRAM = **1920 МБ**
- Суммарный объём VRAM всех трёх серверов = **1920 МБ**

Это ровно **100%**! Все задачи встали как пазл — точно в точку.

Именно поэтому RAM на некоторых серверах не загружена до конца — задачи упёрлись в VRAM, а не в RAM. VRAM — это и есть «узкое горлышко» всей системы.

🔍 Часть 3. Метод ветвей и границ

Это самая важная часть для защиты. Разбираем всё с нуля, включая откуда берётся каждое число.

Сначала важное: данные ручного примера

Чтобы считать руками, взяли **уменьшенную версию** — 4 задачи и 2 сервера.

Серверы в ручном примере:

- **S1:** RAM = 1024 МБ, VRAM = 512 МБ
- **S2:** RAM = 768 МБ, VRAM = 384 МБ

Задачи в ручном примере:

- **Задача 1:** RAM 512 МБ, VRAM 256 МБ, приоритет **8**
- **Задача 2:** RAM 256 МБ, VRAM 128 МБ, приоритет **6**
- **Задача 3:** RAM 1024 МБ, VRAM 512 МБ, приоритет **9**
- **Задача 4:** RAM 128 МБ, VRAM 64 МБ, приоритет **4**

Запомни эти числа — дальше всё будет понятно.

Представь, что ты ищешь клад

Перед тобой карта с развилками. На каждой развилке — два пути. Клад спрятан в одном месте. Ты хочешь найти его **быстро**, не обходя каждую тропинку.

Шаг 1 — Послабление (ЛП-релаксация). Откуда 0.875 и 25.875?

Сначала решаем задачу с **читерством**: разрешаем выполнить задачу не целиком, а дробно — хоть на 87.5%, хоть на 50%.

Компьютер жадный — он хочет набрать максимум приоритетов. Вот что он делает по шагам:

Сначала расставляет задачи, которые точно влезают:

- На S2 ставит задачу 1 (512 МБ) и задачу 2 (256 МБ). Итого: $512 + 256 = 768$ МБ — S2 заполнен полностью.
- На S1 ставит задачу 4 (128 МБ). На S1 остаётся: $1024 - 128 = \mathbf{896 \text{ МБ}}$ свободно.

Теперь хочет втиснуть задачу 3 на S1, но задача 3 весит 1024 МБ, а места только 896:

- $896 \div 1024 = \mathbf{0.875}$
- Вот откуда 0.875 — это просто «сколько задачи 3 физически влезает» в оставшееся место на S1 в дробном мире.

Итоговая сумма приоритетов (откуда 25.875):

- Задача 4 на S1 целиком → приоритет **4**
- Задача 3 на S1 на 87.5% → $9 \times 0.875 = \mathbf{7.875}$
- Задача 1 на S2 целиком → приоритет **8**
- Задача 2 на S2 целиком → приоритет **6**
- Итого: $4 + 7.875 + 8 + 6 = \mathbf{25.875}$ ✓

Что такое «потолок» и зачем он нужен: Дробный мир — это читерство, в реальности нельзя выполнить 87.5% задачи. Но раз **даже с читерством** максимум 25.875, то **без читерства** точно будет не больше. Реальный ответ ≤ 25 . Это и есть потолок — верхняя граница возможного.

Смысл потолка: если мы уже нашли какое-то решение с $Z = 23$, и вдруг попадаем в ветку, где потолок тоже 23 или меньше — туда идти бессмысленно, лучше мы уже не найдём. Ветку отрубаем.

Шаг 2 — Ветвление

Дробная переменная $x_{31} = 0.875$ — это проблема. Выбираем её и делим задачу надвое:

- **Ветка А:** $x_{31} = 1$ — задача 3 точно идёт на сервер 1 (берём целиком)
- **Ветка Б:** $x_{31} = 0$ — задача 3 точно **не** идёт на сервер 1

Решаем каждую ветку отдельно.

Шаг 3 — Рекорд (инкумбент)

Нашли первое рабочее **целое** решение (без дробей, без перегрузки памяти)? Запомнили как **рекорд**.

- В начале рекорд = минус бесконечность (ничего не нашли)
- Нашли $Z = 23 \rightarrow$ рекорд = **23** ?
- Все последующие ветки сравниваем с этим числом

Шаг 4 — Отсечения

Ветка **отрубается** в трёх случаях:

- **По оценке** — потолок ветки $\leq$ рекорду. В лучшем случае не улучшим — незачем идти.
- **По памяти** — задачи физически не помещаются в сервер. Невозможно.
- **По целочисленности** — нашли готовое целое решение, обновляем рекорд и не идём дальше.

Ручной расчёт — разобранный по косточкам

Шаг 1. Корень. Решаем с дробями (как описано выше) $\rightarrow Z = 25.875, x_{31} = 0.875$ — дробная. Ветвимся по ней.

Шаг 2. Ветвь А — жёстко говорим: задача 3 идёт на S1 целиком.

Задача 3 требует RAM 1024 МБ — это ровно весь S1 (у него 1024 МБ). На S1 больше не осталось места — туда никого. Задачи 1, 4 отправляются на S2.

Теперь смотрим что влезает на S2 (768 МБ RAM, 384 МБ VRAM):

- Задача 1: RAM 512, VRAM 256 — влезает ☐
- Задача 2: RAM 256, VRAM 128 — после задачи 1 осталось $768 - 512 = 256$ МБ RAM и $384 - 256 = 128$ МБ VRAM — влезает ☐
- Задача 4: RAM 128 — после задач 1 и 2 RAM занято уже 768 из 768, места нет ☐

Итого выполнили:

- Задача 3 (приоритет **9**) на S1
- Задача 1 (приоритет **8**) на S2
- Задача 2 (приоритет **6**) на S2

$Z = 9 + 8 + 6 = \mathbf{23} \rightarrow$ это первое целое решение $\rightarrow$ **рекорд = 23** ☐

Шаг 3. Ветвь Б — жёстко говорим: задача 3 НЕ идёт на S1.

Случай Б1 — пробуем поставить задачу 3 на S2:

- Задача 3 требует RAM 1024 МБ, а у S2 всего 768 МБ.
- $1024 > 768$ — не влезает физически $\rightarrow$ ☐ **отсечение по памяти.**

Случай Б2 — задача 3 не идёт никуда вообще:

- Мы теряем приоритет 9 от задачи 3.
- Максимум что можем набрать из оставшихся трёх задач: $8 + 6 + 4 = \mathbf{18}$.
- $18 < \text{рекорда } 23 \rightarrow$ даже в лучшем случае мы проиграем $\rightarrow$ ☐ **отсечение по оценке.**

Итог: все ветки либо исследованы, либо отрублены. Рекорд $Z^* = \mathbf{23}$ — это глобальный оптимум. Лучше математически невозможно. ☐

Зачем программа, если есть ручной счёт?

Ручной счёт на 4 задачах занял целую страницу. В реальной задаче 24 переменных. Программа решает это за **2–3 миллисекунды**.

Как устроена программа?

1. **Входные данные** — списки задач и серверов с параметрами
2. **Модель** — создаём 24 выключателя, записываем целевую функцию и 14 ограничений
3. **Решение** — запускаем CBC (мощный солвер на C++), он строит дерево и находит оптимум
4. **Результаты** — выводим куда пошла каждая задача и сколько памяти занято
5. **Графики** — три картинки через Matplotlib, сохраняются в PNG

Почему PuLP и CBC?

- **PuLP** — это как конструктор LEGO: ты собираешь модель на Python, описываешь переменные и правила.
- **CBC** — это профессиональный мастер: берёт твой конструктор и строит решение. Бесплатный, открытый, проверен годами.

🔗 Часть 5. Проверка через ИИ

Задачу дали **Gemini 3.5** на обычном русском языке — просто текстом. Она ответила правильно: $Z^* = 52$, все задачи назначены верно. 🔗

Но ИИ не заменяет программу:

- **Скорость** — программа: 2–3 мс; ИИ: ~10 секунд.
- **Гарантии** — программа всегда даёт точный ответ; ИИ на 100 задачах начнёт «галлюцинировать».
- **Воспроизводимость** — программа каждый раз выдаёт одинаковый результат; ИИ может ответить по-разному.

🔗 Часть 6. Что сказать на защите

«О чём твоя работа?»

Я разработала систему, которая автоматически распределяет задачи обработки видео и фото по серверам так, чтобы выполнить максимально важные задачи и не переполнить память серверов. Для этого использовала метод целочисленного программирования.

«Почему GMMKP?»

Это обобщённый рюкзак. У нас несколько серверов, два типа памяти, и для каждой задачи нужно выбрать конкретный сервер — это три усложнения по сравнению с классическим рюкзаком, что делает задачу NP-трудной.

«Как работает метод ветвей и границ?»

Алгоритм строит дерево решений. Сначала решает задачу с дробными переменными — это даёт верхний потолок. Потом ветвится, запрещая или разрешая назначения. Неперспективные ветки отсекаются, если их потолок хуже уже найденного рекорда.

«Почему не вся RAM использована?»

Потому что узкое горлышко — VRAM. Видеопамять на всех серверах заполнена на 100%. Новые задачи уже некуда ставить по видеопамяти, даже если RAM ещё есть.

«Зачем ручной расчёт, если есть программа?»

Ручной расчёт — это верификация. Мы упростили задачу до 4 задач и 2 серверов, посчитали вручную, и результат совпал с программой. Это доказывает, что математическая модель работает правильно.

? Финальная шпаргалка

- **Суть работы** — распределить 8 задач по 3 серверам максимально выгодно
- **Ограничения** — RAM и VRAM каждого сервера
- **Метод** — целочисленное программирование (ЦЛП)
- **Модель** — GMMKP: рюкзак с двумя ресурсами и тремя рюкзаками
- **Алгоритм** — метод ветвей и границ
- **Программа** — Python + PuLP + CBC
- **Результат** — $Z^* = 52$, идеальный максимум, все задачи выполнены
- **Узкое горлышко** — VRAM загружена на 100%
- **Верификация** — ручной расчёт ($Z^* = 23$) и ИИ Gemini ($Z^* = 52$) — оба совпали ?

Ты справишься. Вся эта математика — просто умный способ решить задачу про выбор задач для серверов. Ты это знаешь лучше, чем думаешь ?